

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT on

Artificial Intelligence (23CS5PCAIN)

Submitted by
Srushti N (1BM24CS424)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)

BENGALURU-560019
Aug 2025 to Dec 2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Artificial Intelligence (23CS5PCAIN)” carried out by **Srushti N (1BM24CS424)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Artificial Intelligence (23CS5PCAIN) work prescribed for the said degree.

Sonika Sharma D Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
--	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	30-9-2024	Implement Tic –Tac –Toe Game Implement vacuum cleaner agent	4-15
2	7-10-2024	Implement 8 puzzle problems using Depth First Search (DFS) Implement Iterative deepening search algorithm	16-26
3	14-10-2024	Implement A* search algorithm	27-40
4	21-10-2024	Implement Hill Climbing search algorithm to solve N-Queens problem	41-48
5	28-10-2024	Simulated Annealing to Solve 8-Queens problem	49-51
6	11-11-2024	Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.	52-57
7	2-12-2024	Implement unification in first order logic	58-63
8	2-12-2024	Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.	64-69
9	16-12-2024	Create a knowledge base consisting of first order logic statements and prove the given query using Resolution	70-76
10	16-12-2024	Implement Alpha-Beta Pruning.	76-81

Github Link:

<https://github.com/SrushtiSunkad630/Artificial-Intelligence>

Implement Tic – Tac – Toe Game Implement vacuum cleaner agent

18/8/05

Page No: _____
Date: ____/____/____

AT Lab - 7 Week - 01

Step 1: Start the game

Step 2: Initialize 3x3 board

Step 3: Check for possibilities where u can put x & set current player = 'x'

Algorithm

Step 1: Start the game

Step 2: Initialize 3x3 board

Step 3: Check for possibilities where u can put x & set current player = 'x'

- Step 4: loop until win or draw
 Step 5: If move is valid, update board
 Step 6: switch the player 1 & player 2 in each game
 Step 7: check, if it is win end the game
 Step 8: check, if it is draw end the game
 Step 9: stop the game

output:

Player x's turn

Enter row (0,1,2) : 0

Enter column (0,1,2) : 0

X		

Player's O's turn

Enter row (0,1,2) : 1

Enter column (0,1,2) : 0

X		
O		

player x's turn

Enter row (0,1 or 2): 1

Enter column (0,1 or 2): 0

X		
0	X	

player's o's turn

Enter row (0,1 or 2): 2

Enter column (0,1 or 2): 2

X		
0	X	
		0

player x's turn

Enter row (0,1 or 2): 0

Enter column (0,1 or 2): 1

X		
0	X	
		0

player x wins

* The game ended on the 5th move

* Winner : player x

* Each move in the Tic-Tac-Toe has a cost of 1

* cost = 5

* because the game ended on the 5th move.

```

|  |
-----
|  |
-----
|  |
-----
Player X's turn.
Enter row (0, 1, or 2): 0
Enter column (0, 1, or 2): 0

```

```

X |  |
-----
|  |
-----
|  |
-----

```

```

Player O's turn.
Enter row (0, 1, or 2): 1
Enter column (0, 1, or 2): 0

```

```

X |  |
-----
O |  |
-----
|  |
-----

```

```

Player X's turn.
Enter row (0, 1, or 2): 1
Enter column (0, 1, or 2): 1

```

```

X |  |
-----
O | X |
-----
|  |
-----

```

```

Player O's turn.
Enter row (0, 1, or 2): 1
Enter column (0, 1, or 2): 2

```

```

X |  |
-----
O | X | O
-----
|  |
-----

```

```

Player X's turn.
Enter row (0, 1, or 2): 2
Enter column (0, 1, or 2): 2

```

```

X |  |
-----
O | X | O
-----
|  | X
-----

```

```

Player X wins!

```

Srushti N
1BM24CS424

Code:

Tic –Tac –Toe Game

```
def print_board(board):
    for row in board:
        print(" | ".join(row))
        print("-" * 9)

def check_winner(board, player):
    # Check rows, columns and diagonals
    for i in range(3):
        if all([cell == player for cell in board[i]]) or \
            all([board[j][i] == player for j in range(3)]):
            return True

    if all([board[i][i] == player for i in range(3)]) or \
        all([board[i][2 - i] == player for i in range(3)]):
        return True

    return False

def is_full(board):
    return all(cell in ['X', 'O'] for row in board for cell in row)

def get_move(player):
    while True:
        try:
            move = input(f"Player {player}, enter your move (row and column: 1 1): ")
            row, col = map(int, move.split())
            if row in [1, 2, 3] and col in [1, 2, 3]:
                return row - 1, col - 1
            else:
                print("Invalid input. Enter numbers between 1 and 3.")
        except ValueError:
            print("Invalid input. Enter two numbers separated by space.")

def play_game():
    board = [[" " for _ in range(3)] for _ in range(3)]
    current_player = "X"

    while True:
        print_board(board)
        row, col = get_move(current_player)
```

```

if board[row][col] != " ":
    print("That spot is taken. Try again.")
    continue

board[row][col] = current_player

if check_winner(board, current_player):
    print_board(board)
    print(f"Player {current_player} wins!")
    break

if is_full(board):
    print_board(board)
    print("It's a draw!")
    break

current_player = "O" if current_player == "X" else "X"

if __name__ == "__main__":
    play_game()

```

Output:

```
| | |
-----
| | |
-----
| | |
-----
Player X, enter your move (row and column: 1 1): 1 1
X | | |
-----
| | |
-----
| | |
-----
Player O, enter your move (row and column: 1 1): 1 2
X | O | |
-----
| | |
-----
| | |
-----
Player X, enter your move (row and column: 1 1): 1 3
X | O | X
-----
| | |
-----
| | |
-----
Player O, enter your move (row and column: 1 1): 2 2
X | O | X
-----
| O | |
-----
| | |
-----
Player X, enter your move (row and column: 1 1): 3 3
X | O | X
-----
| O | |
-----
| | | X
-----
Player O, enter your move (row and column: 1 1): 3 1
X | O | X
-----
| O | |
-----
O | | | X
-----
Player X, enter your move (row and column: 1 1): 2 3
X | O | X
-----
| O | X
-----
O | | | X
-----
Player X wins!
```


Vacuum Cleaner:

Algorithm:

Page No:
 Date:

Vacuum Cleaner Agent

Step 1: Consider the two location A & B
Step 2: Start the vacuum cleaner agent
Step 3: check the status (clean or dirty)
in the location A, record the direction, else off the vacuum cleaner.
Step 4: Ask the user to:
* to clean the room
* to stay in the room
* to move to next location
Step 5: If the user select
1, clean the location
2, then stay in the room
3, or move to next location.
Step 6: For the location B, start from or Repeat from step 3.
Step 7: Stop the vacuum cleaner agent

Cost Calculation:
 $O(b^d)$
 $b = 4$
 $d = 2$
 $O(4)^2 = O(16)$

Output:
Enter state of A (0 for clean, 1 for dirty):
Enter state of B (0 for clean, 1 for dirty):
Enter location (A or B): A
cleaned A
Moving vacuum right
cleaned B
Cost = 0
'A': 0, 'B': 03

Enter state of A (0 for clean, 1 for dirty): 1
 Enter state of B (0 for clean, 1 for dirty): 0
 Enter location (A or B): A

Cleaned A

Moving vacuum right

B is Clean

Cost: 1

<'A': 0, 'B': 0>

OUTPUT

```
Enter state of A (0 for clean, 1 for dirty): 1
Enter state of B (0 for clean, 1 for dirty): 1
Enter location (A or B): A
Cleaned A.
Moving vacuum right
Cleaned B.
Cost: 2
{'A': 0, 'B': 0}
|
```

```
Enter state of A (0 for clean, 1 for dirty): 0
Enter state of B (0 for clean, 1 for dirty): 0
Enter location (A or B): B
Turning vacuum off
Cost: 0
{'A': 0, 'B': 0}
```

```
Enter state of A (0 for clean, 1 for dirty): 1
Enter state of B (0 for clean, 1 for dirty): 0
Enter location (A or B): A
Cleaned A.
Moving vacuum right
B is clean
Cost: 1
{'A': 0, 'B': 0}
```

```
Enter state of A (0 for clean, 1 for dirty): 0
Enter state of B (0 for clean, 1 for dirty): 1
Enter location (A or B): B
Cleaned B.
Moving vacuum left
A is clean
Cost: 1
{'A': 0, 'B': 0}
```

Srushti N
1BM24CS424

Code:

```
def vacuum_simulation():  
    cost = 0  
  
    # Get initial states and location  
    state_A = int(input("Enter state of A (0 for clean, 1 for dirty): "))  
    state_B = int(input("Enter state of B (0 for clean, 1 for dirty): "))  
    location = input("Enter location (A or B): ").upper()  
  
    # Vacuum operation loop  
    while True:  
        if location == 'A':  
            if state_A == 1:  
                print("Cleaning A.")  
                state_A = 0  
                cost += 1  
            elif state_B == 1:  
                print("Moving vacuum right")  
                location = 'B'  
                cost += 1  
            else:  
                print("Turning vacuum off")  
                break  
  
        elif location == 'B':  
            if state_B == 1:  
                print("Cleaning B.")  
                state_B = 0  
                cost += 1  
            elif state_A == 1:
```

```
        print("Moving vacuum left")
        location = 'A'
        cost += 1
    else:
        print("Turning vacuum off")
        break
    print(f"Cost: {cost}")
    print(f'{{"A": {state_A}, "B": {state_B}}}')

```

vacuum_simulation()

OUTPUT

```
Enter state of A (0 for clean, 1 for dirty): 1
Enter state of B (0 for clean, 1 for dirty): 0
Enter location (A or B): A
Cleaning A.
Turning vacuum off
Cost: 1
{'A': 0, 'B': 0}
```

Implement 8 puzzle problems using Depth First Search (DFS)

[illegible]

Algorithm using DFS:

- Step 1: start with the initial state of the puzzle
- Step 2: check if the puzzle is solvable by counting inversions. if not stop
- Step 3: push it into a stack & mark it visited
- Step 4: while the stack is not empty:
 - pop the top state
 - if it is the goal, stop
 - otherwise, generate all pos next states & push unvisited ones into the stack
- Step 5: Trace back the path to get the solution (DFS may not give shortest path)

Algorithm using BFS:

- Step 1: start with the initial state
- Step 2: put it in a queue & mark it visited
- Step 3: while the queue is not empty:
 - Remove the front state
 - if it is the goal, stop
 - otherwise, generate all pos next states & add unvisited ones to the queue
- Step 4: Trace back to the path to get the solution. (BFS gives shortest path).

DFS

2	8	3
1	6	4
7		5

↓ down

2	8	3
1		4
7	6	5

↓ down

2	8	3
1	8	4
7	6	5

→ right

	2	3
1	8	4
7	6	5

up

1	2	3
	8	4
7	6	5

left

1	2	3
8		4
7	6	5

output for BFS

output for DFS

solution found in 5 moves:

sol'n found in 5 moves

2	8	3
1	6	4
7	0	5

initial

2	8	3
1	6	4
7	0	5

initial

↓

2	8	3
1	0	4
7	6	5

↓

2	8	3
1	0	4
7	6	5

↓

2	0	3
1	8	4
7	6	5

↓

2	0	3
1	8	4
7	6	5

↓

0	2	3
1	8	4
7	6	5

↓

0	2	3
1	8	4
7	6	5

↓

1	2	3
0	8	4
7	6	5

↓

1	2	3
0	8	4
7	6	5

↓

1	2	3
8	0	4
7	6	5

goal

↓

1	2	3
8	0	4
7	6	5

goal

Iterative Deepening Search (IDS) or Iterative Deepening Depth First Search (IDDFS)

1	2	3
4	0	5
6	7	8

initial

1	2	3
4	5	0
6	7	8

goal

sol'n found on depth 1

Right →

3	0	8
2	5	7
1	6	4

Algorithm :

left →

3	5	8
2	4	7
1	0	6

Step 1: Set depth limit = 0

Step 2: Perform Depth-limited DFS from the initial state up to the current depth limit

Step 3: if the goal is found during the path DFS, stop & return the path

Step 4: if not found, increase the depth limit by 1

Step 5: Repeat steps 2-4 until the goal is found or max depth is reached.

3	5	8
2	4	7
1	0	6

Limit 0

down →

3	5	8
2	7	4
1	6	0

up →

3	5	8
0	2	7
1	4	6

Limit -1

Output:

searching with depth limit = 0

searching with depth limit = 1

Solution found in 1 moves

step 0:

1 2 3

4 0 5

6 7 8

Step 1:

1 2 3

4 5 0

6 7 8

Code:

Using DFS 8 puzzle without heuristic

Goal state

```
goal = ((1, 2, 3),
        (8, 0, 4),
        (7, 6, 5))
```

Moves: Up, Down, Left, Right

```
moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

```
def get_neighbors(state):
```

```
    # Find the empty tile (0)
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            if state[i][j] == 0:
```

```
                x, y = i, j
```

```
                break
```

```
neighbors = []
```

```
for dx, dy in moves:
```

```
    nx, ny = x + dx, y + dy
```

```
    if 0 <= nx < 3 and 0 <= ny < 3:
```

```
        # Swap empty tile with adjacent tile
```

```
        new_state = [list(row) for row in state]
```

```
        new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
```

```
        neighbors.append(tuple(tuple(row) for row in new_state))
```

```
return neighbors
```

```
def dfs_limited(start, depth_limit):
```

```
    stack = [(start, [start])]
```

```
    visited = set([start])
```

```
while stack:
```

```
    current, path = stack.pop()
```

```
    if current == goal:
```

```
        return path
```

```
    if len(path) - 1 >= depth_limit: # already reached depth limit
```

```
        continue
```

```
    for neighbor in get_neighbors(current):
```

```
        if neighbor not in visited:
```

```
            visited.add(neighbor)
```

```
            stack.append((neighbor, path + [neighbor]))
```

```
return None
```

```

# Example start state
start = ((2, 8, 3),
        (1, 6, 4),
        (7, 0, 5))

solution_path = dfs_limited(start, depth_limit=5)

if solution_path:
    print(f'Solution found in {len(solution_path) - 1} moves:')
    for state in solution_path:
        for row in state:
            print(row)
        print()
else:
    print("No solution found within 5 moves.")

```

OUTPUT

```

Solution found in 5 moves:
(2, 8, 3)
(1, 6, 4)
(7, 0, 5)

(2, 8, 3)
(1, 0, 4)
(7, 6, 5)

(2, 0, 3)
(1, 8, 4)
(7, 6, 5)

(0, 2, 3)
(1, 8, 4)
(7, 6, 5)

(1, 2, 3)
(0, 8, 4)
(7, 6, 5)

(1, 2, 3)
(8, 0, 4)
(7, 6, 5)

```


Iterative Deepening Search(IDS) or Iterative Deepening Depth First Search(IDDFS)

```
from copy import deepcopy

GOAL_STATE = [
    [1, 2, 3],
    [4, 5, 0],
    [6, 7, 8]
]

# Possible moves of the blank (0) tile: up, down, left, right
MOVES = [(-1, 0), (1, 0), (0, -1), (0, 1)]

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

def is_goal(state):
    return state == GOAL_STATE

def get_neighbors(state):
    neighbors = []
    x, y = find_blank(state)

    for dx, dy in MOVES:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = deepcopy(state)
            # Swap blank with neighbor
```

```

        new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
        neighbors.append(new_state)
    return neighbors

def dfs(state, depth, limit, path, visited):
    if is_goal(state):
        return path + [state]
    if depth == limit:
        return None

    for neighbor in get_neighbors(state):
        # To avoid cycles, do not revisit states in current path
        if neighbor not in visited:
            result = dfs(neighbor, depth + 1, limit, path + [state], visited + [neighbor])
            if result is not None:
                return result
    return None

def iterative_deepening_search(initial_state, max_depth=50):
    for depth_limit in range(max_depth):
        print(f'Searching with depth limit = {depth_limit}')
        result = dfs(initial_state, 0, depth_limit, [], [initial_state])
        if result is not None:
            return result
    return None

def print_state(state):
    for row in state:
        print(' '.join(str(x) for x in row))
    print()

```

```

if __name__ == "__main__":
    initial_state = [
        [1, 2, 3],
        [4, 0, 5],
        [6, 7, 8]
    ]

    solution = iterative_deepening_search(initial_state)

    if solution:
        print(f'Solution found in {len(solution)-1} moves!')
        for step, state in enumerate(solution):
            print(f'Step {step}:')
            print_state(state)
    else:
        print("No solution found.")

```

OUTPUT

```

Searching with depth limit = 0
Searching with depth limit = 1
Solution found in 1 moves!
Step 0:
1 2 3
4 0 5
6 7 8

Step 1:
1 2 3
4 5 0
6 7 8

```

Implement A* search algorithm
Manhattan:

Lab-4

Page No. _____
Date: 08 09 2025

Apply A* Algorithm

Misplaced Tiles Manhattan distance

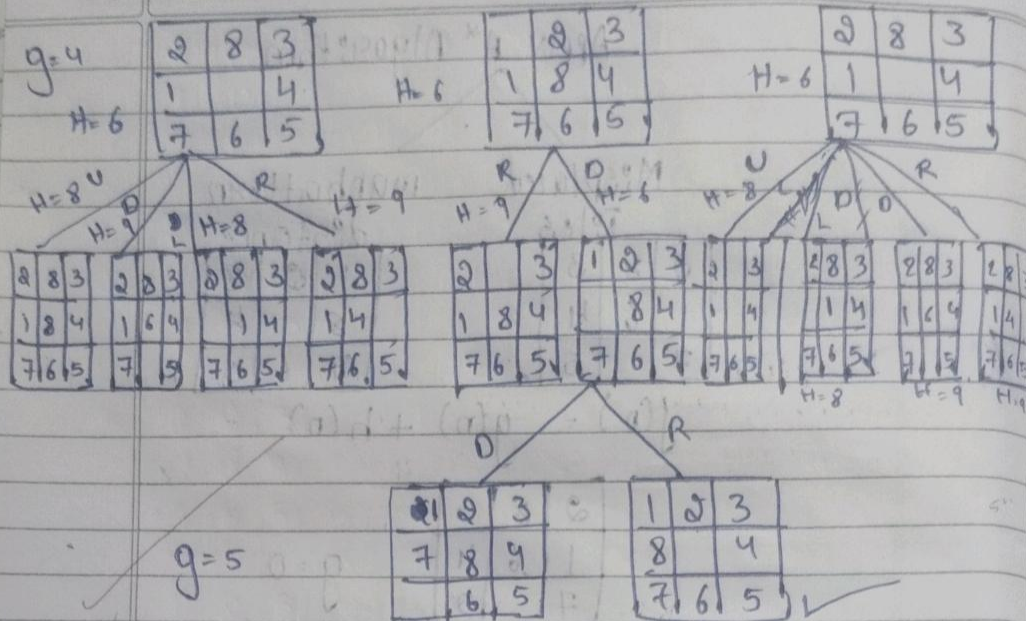
$f(n) = g(n) + h(n)$

$g=0$

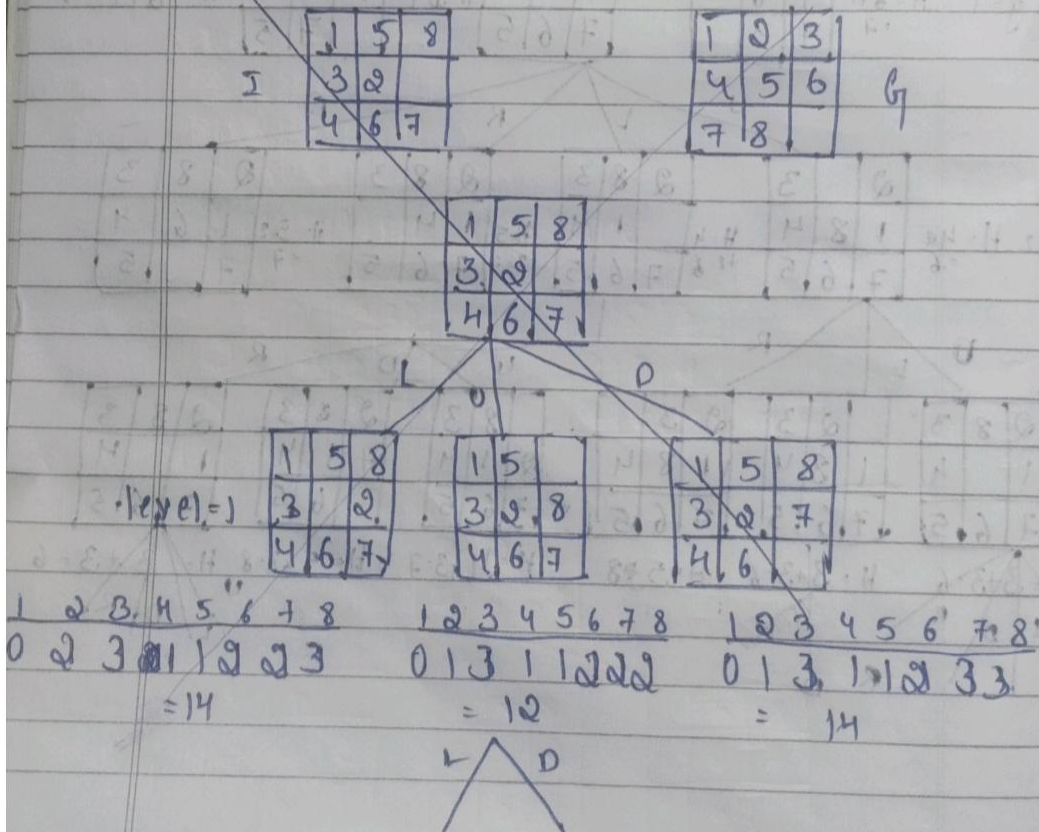
$g=1$ $H=6+1=7$

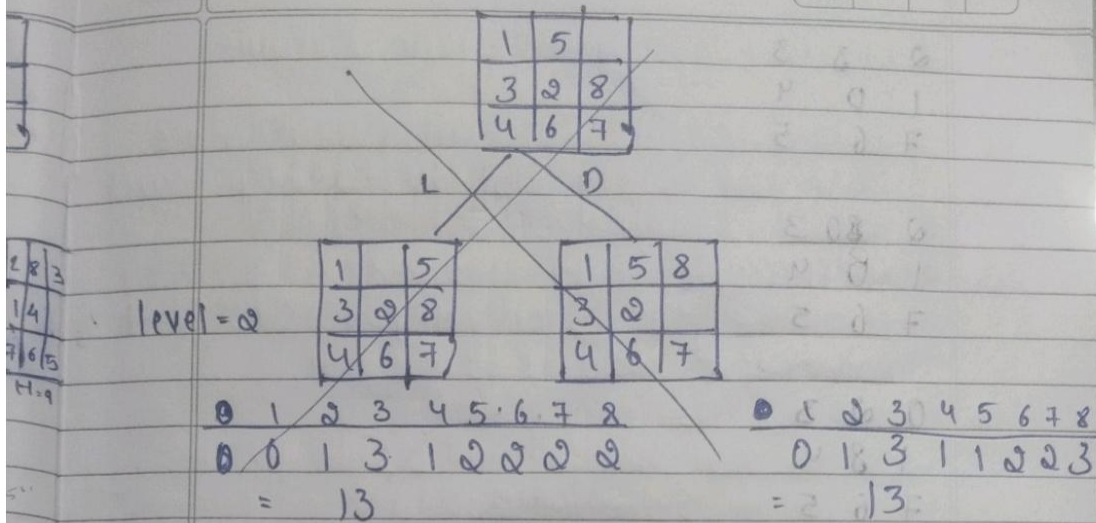
$g=2$ $H=4+2=6$

$H=3+3=6$ $H=3+3=6$ $H=5+3=8$ $H=4+3=7$ $H=5+3=8$ $H=3+3=6$



Q. Manhattan distance.





Algo for (Misplaced Tiles)

1. Start $\rightarrow$ put initial state in OPEN ($f = g + h$)
2. Pick state with smallest
3. If goal stop
4. Expand neighbors
5. for each neighbor
 - $g := \text{parent.g} + 1$
 - $h = \text{misplaced tiles}$
 - $f = g + h$
 - add / update in Open
6. Repeat until goal found or open empty.

Output for Misplaced Tiles.

0 8 3
1 6 4
7 0 5

(2, 8, 3)
(1, 6, 4)
(7, 0, 5)

(2, 8, 3)
(1, 0, 4)
(7, 6, 5)

(2, 0, 3)
(1, 8, 4)
(7, 6, 5)

(0, 2, 3)
(1, 8, 4)
(7, 6, 5)

(1, 2, 3)
(0, 8, 4)
(7, 6, 5)

(1, 2, 3)
(8, 0, 4)
(7, 6, 5)

Srushti N
1BM24CS424

2 8 3
1 0 4
7 6 5

2 8 0 3
1 8 4
7 6 5

0 2 3
1 8 4
7 6 5

1 2 3
0 8 4
7 6 5

Manhattan distance

2 8 38
31 4
7 0 45

1 2 3
8 4
7 8 5

2 8 38
31 4
7 0 45

2 3 4 5 6 7 8
1 0 0 1 1 2
= 4

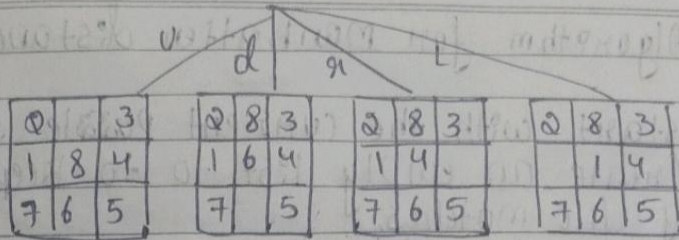
1 5
3 2 8
4 6 7

1 5 8
3 2
4 6 7

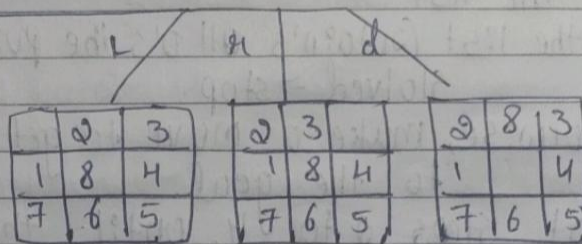
1 5 8
3 2 7
4 6

1 2 3 4 5 6 7 8
1 1 0 0 1 1 0 2
= 6

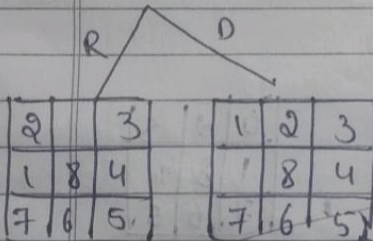
1 1 3 4 5 6 7 8
1 1 0 0 0 0 0 2 = 6



1 2 3 4 5 6 7 8 1 2 3 4 5 6 7 8 1 2 3 4 5 6 7 8 1 2 3 4 5 6 7 8
 1 1 0 0 0 0 0 1 1 1 0 0 0 1 0 2 1 1 0 1 0 0 0 2 2 1 0 0 0 0 0 2
 = 3 = 5 = 5 = 5

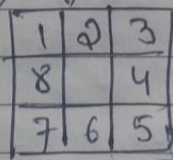


1 2 3 4 5 6 7 8 1 2 3 4 5 6 7 8 1 2 3 4 5 6 7 8
 1 0 0 0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 0 0 0 0 0 2
 = 2 = 4 = 4

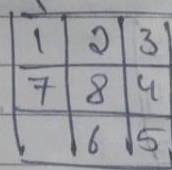
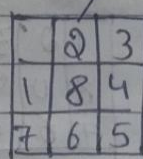


1 2 3 4 5 6 7 8
 0 0 0 0 0 0 0 1 = 1

1 2 3 4 5 6 7 8 = 3
 1 1 0 0 0 0 0 1



1 2 3 4 5 6 7 8
 = 0 0 0 0 0 0 0 6



7 8
 0 2

Algorithm for manhattan distance

1. Start with the current puzzle state.
2. Create an empty list to keep track of file matches.
3. For each file from 1 to 8:
 - if the file is in correct pos, add 0 to the list.
 - otherwise, add 1 to the list.
4. Check the list:
 - if the list contains all 0's the puzzle is solved → stop.
 - otherwise, make a move to get closer to the goal.
5. Repeat steps 2 to 4 until the puzzle is solved.

Output:

Solution path:

2	8	3
1	6	4
7	0	5

↓

2	8	3
1	0	4
7	6	5

↓

2	0	3
1	8	4
7	6	5

↓

0	2	3
1	8	4
7	6	5

↓

1	2	3
0	8	4
7	6	5

↓

1	2	3
8	0	4
7	6	5

Solution path:

(2, 8, 3)

(1, 6, 4)

(7, 0, 5)

(2, 8, 3)

(1, 0, 4)

(7, 6, 5)

(2, 0, 3)

(1, 8, 4)

(7, 6, 5)

(0, 2, 3)

(1, 8, 4)

(7, 6, 5)

(1, 2, 3)

(0, 8, 4)

(7, 6, 5)

(1, 2, 3)

(8, 0, 4)

(7, 6, 5)

Srushti N
1BM24CS424

Code:

Misplace Tiles:

```
import heapq

# Goal state
goal = ((1, 2, 3),
        (8, 0, 4),
        (7, 6, 5))

# Moves: Up, Down, Left, Right
moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

# Heuristic: Misplaced tiles
def misplaced_tiles(state):
    count = 0
    for i in range(3):
        for j in range(3):
            if state[i][j] != 0 and state[i][j] != goal[i][j]:
                count += 1
    return count

# Find blank position (0)
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# Generate neighbors
def get_neighbors(state):
    neighbors = []
```

```

x, y = find_blank(state)

for dx, dy in moves:
    nx, ny = x + dx, y + dy
    if 0 <= nx < 3 and 0 <= ny < 3:
        new_state = [list(row) for row in state]
        new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
        neighbors.append(tuple(tuple(row) for row in new_state))

return neighbors

# A* Search
def astar(start):
    pq = []
    heapq.heappush(pq, (misplaced_tiles(start), 0, start, []))
    visited = set()

    while pq:
        f, g, state, path = heapq.heappop(pq)
        if state == goal:
            return path + [state]
        if state in visited:
            continue
        visited.add(state)
        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                new_g = g + 1
                new_f = new_g + misplaced_tiles(neighbor)
                heapq.heappush(pq, (new_f, new_g, neighbor, path + [state]))

    return None

```

```
# Example usage
start_state = ((2, 8, 3),
               (1, 6, 4),
               (7, 0, 5))

solution = astar(start_state)

# Print solution path
for step in solution:
    for row in step:
        print(row)
    print("-----")
```

OUTPUT

```
(2, 8, 3)
(1, 6, 4)
(7, 0, 5)
-----
(2, 8, 3)
(1, 0, 4)
(7, 6, 5)
-----
(2, 0, 3)
(1, 8, 4)
(7, 6, 5)
-----
(0, 2, 3)
(1, 8, 4)
(7, 6, 5)
-----
(1, 2, 3)
(0, 8, 4)
(7, 6, 5)
-----
(1, 2, 3)
(8, 0, 4)
(7, 6, 5)
-----
```

```

Manhattan:

import heapq

goal = ((1, 2, 3),
        (8, 0, 4),
        (7, 6, 5))

moves = [(-1, 0), (1, 0), (0, -1), (0, 1)] # Up, Down, Left, Right

def manhattan_distance(state):
    distance = 0
    for i in range(3):
        for j in range(3):
            value = state[i][j]
            if value != 0:
                # goal position of this tile
                goal_x = (value - 1) // 3
                goal_y = (value - 1) % 3
                distance += abs(i - goal_x) + abs(j - goal_y)
    return distance

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

def get_neighbors(state):
    neighbors = []
    x, y = find_blank(state)
    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]

```



```

        new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
        neighbors.append(tuple(tuple(row) for row in new_state))

    return neighbors

def astar(start):
    pq = []
    heapq.heappush(pq, (manhattan_distance(start), 0, start, [])) # (f, g, state, path)
    visited = set()

    while pq:
        f, g, state, path = heapq.heappop(pq)
        if state == goal:
            return path + [state]
        if state in visited:
            continue
        visited.add(state)
        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                new_g = g + 1
                new_f = new_g + manhattan_distance(neighbor)
                heapq.heappush(pq, (new_f, new_g, neighbor, path + [state]))

    return None

start_state = ((2, 8, 3),
               (1, 6, 4),
               (7, 0, 5))

solution = astar(start_state)

if solution is None:
    print("No solution found.")
else:
    print("Solution path:")

```

```
for step in solution:
```

```
    for row in step:
```

```
        print(row)
```

```
    print("-----")
```

OUTPUT:

```
Solution path:
```

```
(2, 8, 3)
```

```
(1, 6, 4)
```

```
(7, 0, 5)
```

```
-----
```

```
(2, 8, 3)
```

```
(1, 0, 4)
```

```
(7, 6, 5)
```

```
-----
```

```
(2, 0, 3)
```

```
(1, 8, 4)
```

```
(7, 6, 5)
```

```
-----
```

```
(0, 2, 3)
```

```
(1, 8, 4)
```

```
(7, 6, 5)
```

```
-----
```

```
(1, 2, 3)
```

```
(0, 8, 4)
```

```
(7, 6, 5)
```

```
-----
```

```
(1, 2, 3)
```

```
(8, 0, 4)
```

```
(7, 6, 5)
```

```
-----
```

Program 4

Implement Hill Climbing search algorithm to solve N-Queens problem

Algorithm:

Week - 4

Page No.
 Date 15 09 25

→ Implement Hill climbing search algorithm to solve N-Queens problem.

Initial input board:

1	2	3	4
			Q
	Q		
		Q	
Q			

(row, col)
(1, 4) (2, 2) (3, 3) (4, 1)

Step 1: initial state

			Q
	Q		
		Q	
Q			

Attacking pairs:
(2, 2) ↔ (3, 3)
(1, 4) ↔ (4, 1)
both diagonal
cost = 2

Step 2:

			Q
Q	Q		Q
		Q	
Q			

Attacking pairs:
(1, 4) ↔ (1, 2)
(2, 2) ↔ (2, 4)
(2, 2) ↔ (3, 3)
(2, 4) ↔ (3, 3)
cost = 3

Step 3:

		Q	
	Q		Q
Q			

A.P ⇒
(1, 3) ↔ (2, 2)
(1, 3) ↔ (2, 4)
(2, 2) ↔ (2, 4)
cost = 3

step 4:

			Q
	Q	Q	
Q			

A.P. $\Rightarrow$
 $(2,4) \leftrightarrow (3,3)$
 $(3,2) \leftrightarrow (4,1)$
 $(3,2) \leftrightarrow (3,3)$
 cost = 3

step 5:

	Q		
Q			Q
Q			
		Q	

A.P. $\Rightarrow$
 no attacking pairs
 (no diagonal, no same
 row & column)

Algorithm

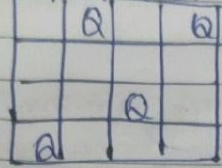
1. start with one queen in each column (initial board)
2. calculate cost(h) = no of attacking queen pairs
3. for each column, move the queen to every other row & compute new cost
4. choose the move that gives the lowest cost (best neighbor)
5. if best cost < cur cost, move queen there & repeat step 2.
6. if no neighbor has lowest cost, STOP.

o/p: $\Rightarrow$ (6, 8)

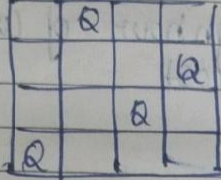
initial state (cost = 2):

			Q
	Q		
		Q	
Q			

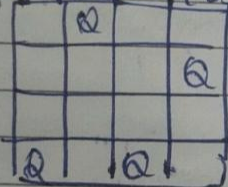
Step 1: Cost = 2



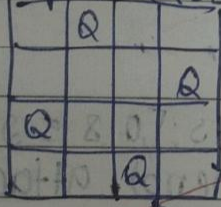
Step 2: Cost = 1



Step 3: (Cost = 1)



Step 4: (Cost = 0)



Soln found in 4 steps.

Initial State (cost=2):

```
. . . Q
. Q . .
. . Q .
Q . . .
```

Step 1 (cost=2):

```
. Q . Q
. . . .
. . Q .
Q . . .
```

Step 2 (cost=1):

```
. Q . .
. . . Q
. . Q .
Q . . .
```

Step 3 (cost=1):

```
. Q . .
. . . Q
. . . .
Q . Q .
```

Step 4 (cost=0):

```
. Q . .
. . . Q
Q . . .
. . Q .
```

Solution found in 4 steps 

Code:

```
import random
```

```
def compute_cost(state):  
    n = len(state)  
    cost = 0  
    for i in range(n):  
        for j in range(i + 1, n):  
            if state[i] == state[j]:          # same row  
                cost += 1  
            elif abs(state[i] - state[j]) == abs(i - j): # same diagonal  
                cost += 1  
    return cost
```

```
def get_neighbors(state):  
    neighbors = []  
    n = len(state)  
    for col in range(n):    # pick a column  
        for row in range(n): # try moving queen in this column to another row  
            if row != state[col]:  
                new_state = state.copy()  
                new_state[col] = row  
                neighbors.append(new_state)  
    return neighbors
```

```
def print_board(state):  
    n = len(state)
```

```

for r in range(n):
    line = ""
    for c in range(n):
        line += "Q " if state[c] == r else ". "
    print(line)
print("")

def hill_climb(initial_state, max_sideways=50):
    current = initial_state
    current_cost = compute_cost(current)
    steps = 0
    sideways_moves = 0

    print("Initial State (cost={}):".format(current_cost))
    print_board(current)

    while True:
        neighbors = get_neighbors(current)
        costs = [compute_cost(n) for n in neighbors]
        min_cost = min(costs)

        if min_cost > current_cost:
            # no better neighbor -> stop
            break

        # pick one of the best neighbors randomly
        best_neighbors = [n for n, c in zip(neighbors, costs) if c == min_cost]
        next_state = random.choice(best_neighbors)
        next_cost = compute_cost(next_state)

```

```

# handle sideways moves
if next_cost == current_cost:
    if sideways_moves >= max_sideways:
        break
    else:
        sideways_moves += 1
else:
    sideways_moves = 0

current = next_state
current_cost = next_cost
steps += 1

print("Step {} (cost={}):".format(steps, current_cost))
print_board(current)

if current_cost == 0:
    print("Solution found in {} steps ☒".format(steps))
    return current

print("Local minimum reached (cost= {}) ✗".format(current_cost))
return current

initial_state = [3, 1, 2, 0]

final = hill_climb(initial_state, max_sideways=10)

```

OUTPUT:

Initial State (cost=2):

```
. . . Q
. Q . .
. . Q .
Q . . .
```

Step 1 (cost=2):

```
. . . Q
Q Q . .
. . Q .
. . . .
```

Step 2 (cost=1):

```
. . . Q
Q . . .
. . Q .
. Q . .
```

Step 3 (cost=1):

```
. . Q Q
Q . . .
. . . .
. Q . .
```

Step 4 (cost=0):

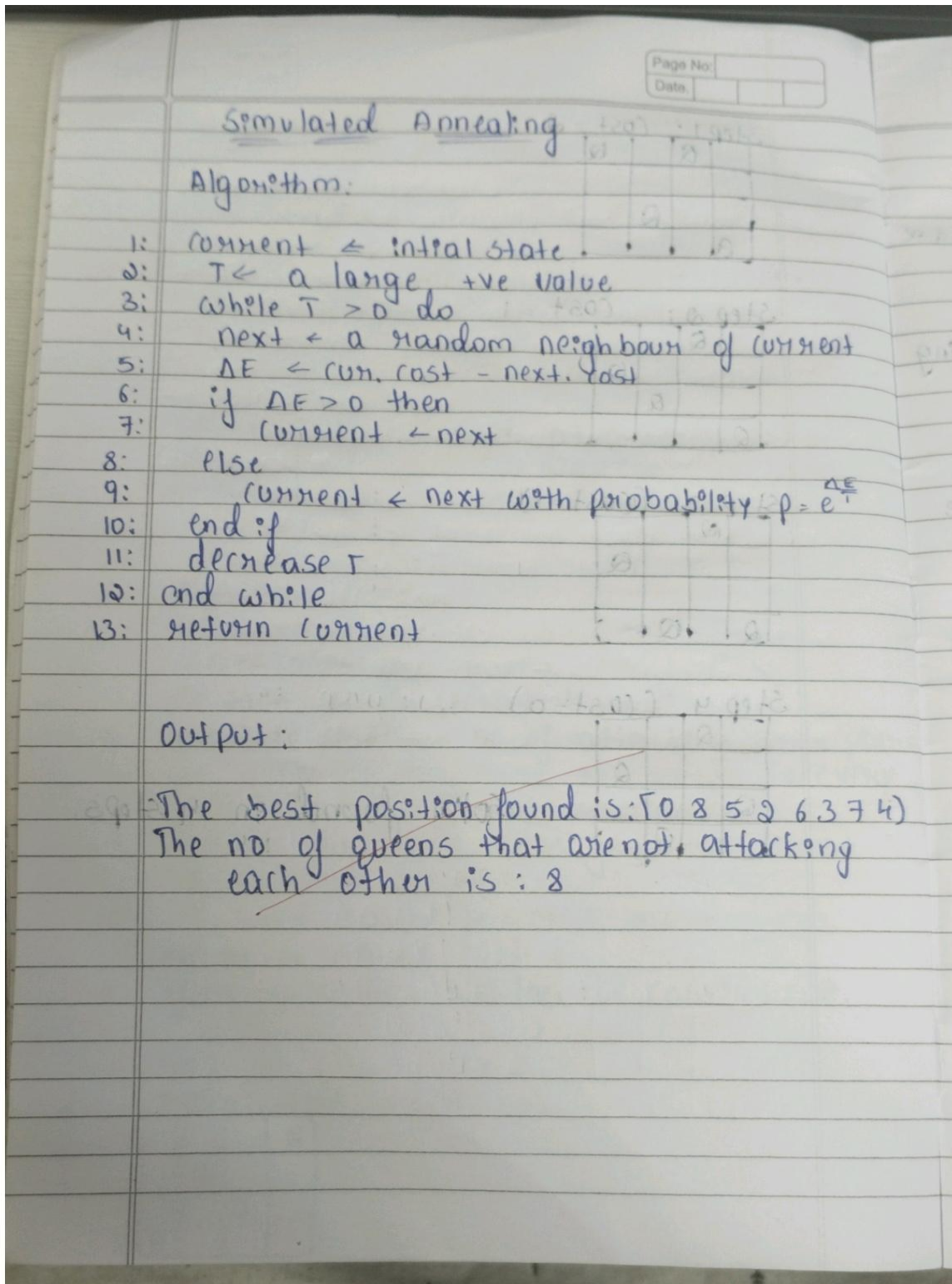
```
. . Q .
Q . . .
. . . Q
. Q . .
```

Solution found in 4 steps 

Program 5

Simulated Annealing to Solve 8-Queens problem

Algorithm:



```
The best position found is: [6, 3, 1, 7, 4, 2, 0, 5]  
The number of queens that are not attacking each other is: 8
```

Code:

```
from scipy.optimize import dual_annealing
import numpy as np

def queens_max(x):
    cols = np.round(x).astype(int)
    n = len(cols)

    if len(set(cols)) < n:
        return 1e6
    attacks = 0
    for i in range(n):
        for j in range(i + 1, n):
            if abs(i - j) == abs(cols[i] - cols[j]):
                attacks += 1
    return attacks

n = 8
bounds = [(0, n - 1)] * n
result = dual_annealing(queens_max, bounds)

best_cols = np.round(result.x).astype(int).tolist()
not_attacking = n

print(f"The best position found is: {best_cols}")
print(f"The number of queens that are not attacking each other is: {not_attacking}")
```

OUTPUT:

```
The best position found is: [7, 4, 6, 1, 3, 5, 0, 2]
The number of queens that are not attacking each other is: 8
```

Program 6

Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.

Algorithm:

Page No:
 Date: 22 09 25

Lab
propositional logic

Implementation of truth-table enumeration algo for deciding propositional entailment
i.e. create a knowledge base using propositional logic & show that the given query entails the knowledge base or not.

Truth table for Connectives:

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$
false	false	true	false	false	true
false	true	true	false	true	false
true	false	false	false	true	false
true	true	false	true	true	true

a. Propositional inference: Enumeration method
Example: $\alpha = A \vee B$ KB = $(A \vee C) \wedge (B \vee C)$

checking that $KB \models \alpha$

A	B	C	$A \vee C$	$B \vee C$	KB	α
false	false	false	false	false	false	false
false	false	true	true	true	true	true
false	true	false	false	true	false	false
false	true	true	true	true	true	true
true	false	false	true	false	false	false
true	false	true	true	true	true	true
true	true	false	true	true	true	true
true	true	true	true	true	true	true

KB $\models \alpha$ holds (KB entails α)

Algorithm

→ List all variables

* Find the symbols that appear in KB & α

* Example: A, B, C

→ Try every possibility

* Each symbol can be True or False

* So we test all combinations (like filling a truth table)

→ Check KB

* For each combination, see if KB is true

→ Check α

* if KB is true, then α must also be true

* if KB is false, we don't care about

α in that row

→ Final decision

* if in all cases where KB is true, α

is also true $\rightarrow$ KB entails α

* if in any case KB is true but α is

false $\rightarrow$ KB does not entail α

a. Consider $S \in T$ as variables & following relations $\rightarrow$

a: $S \vee T$

b: $(S \wedge T)$

c: $T \vee \neg T$

Write truth table & show whether

(i) a entails b

(ii) a entails c

			on	And	
S	T	$\neg T$	$S \vee T$	$S \wedge T$	$T \vee \neg T$
0	0	1	0	0	1
0	1	0	1	0	1
1	0	1	1	0	1
1	1	0	1	1	1

Answer: a entails b $\Rightarrow$ not holds
a entails c $\Rightarrow$ holds.

889
22/9/2025


```
Enter alpha (example: A | B): ( A | B)
Enter KB (example: (A | C) & (B | ~C)): (A | C) & (B | ~C)
```

```
Truth Table for  $\alpha = (A | B)$ ,  $KB = (A | C) \& (B | \sim C)$ 
```

A	B	C	α	KB
False	False	False	False	False
False	False	True	False	False
False	True	False	True	False
False	True	True	True	True
True	False	False	True	True
True	False	True	True	False
True	True	False	True	True
True	True	True	True	True

```
KB |=  $\alpha$  holds (KB entails  $\alpha$ )
```

Srushti N
1BM24CS424

Code:

```
import itertools

from sympy import symbols, sympify

A, B, C = symbols('A B C')

alpha_input = input("Enter alpha (example: A | B): ")
kb_input = input("Enter KB (example: (A | C) & (B | ~C)): ")

alpha = sympify(alpha_input, evaluate=False)
kb = sympify(kb_input, evaluate=False)

GREEN = "\033[92m"
RESET = "\033[0m"

print(f"\nTruth Table for  $\alpha = \{alpha\_input\}$ , KB =  $\{kb\_input\}$ \n")
print(f"{'A':<6} {'B':<6} {'C':<6} {' $\alpha$ ':<10} {'KB':<10}")

entailed = True

for values in itertools.product([False, True], repeat=3):
    subs = {A: values[0], B: values[1], C: values[2]}
    alpha_val = alpha.subs(subs)
    kb_val = kb.subs(subs)

    alpha_str = f"{GREEN}{alpha_val}{RESET}" if kb_val else str(alpha_val)
    kb_str = f"{GREEN}{kb_val}{RESET}" if kb_val else str(kb_val)

    print(f"{'':<6} {'':<6} {'':<6} {'':<10} {'':<10}"
          f"{'alpha_str':<10} {'kb_str':<10}")
```

```
if kb_val and not alpha_val:
```

```
    entailed = False
```

```
if entailed:
```

```
    print(f"\n KB  $\models \alpha$  holds (KB entails  $\alpha$ )\n")
```

```
else:
```

```
    print(f"\n KB does NOT entail  $\alpha$ \n")
```

OUTPUT:

```
Enter alpha (example: A | B): A|B
Enter KB (example: (A | C) & (B | ~C)): (A | C) & (B | ~C)

Truth Table for  $\alpha = A|B$ , KB =  $(A | C) \& (B | \sim C)$ 

A      B      C       $\alpha$       KB
False False False False      False
False False True  False      False
False True  False True       False
False True  True  True       True
True  False False True       True
True  False True  True       False
True  True  False True       True
True  True  True  True       True

KB  $\models \alpha$  holds (KB entails  $\alpha$ )
```

Program 7

Implement unification in first order logic

Algorithm:

Date. 13 10 25

Week - 7
Unification Algorithm

Solve the following

1) Find Most general Unifier (MGU) of $\{P(b, x, f(g(z))), P(z, f(y), f(y))\}$

$\Rightarrow$

- $b = z \rightarrow z = b$
- $x = f(y)$
- $f(g(z)) = f(y) \rightarrow g(z) = y$

Substitution

- $z \rightarrow b$
- $x \rightarrow f(y)$
- $y \rightarrow g(z)$

MGU: $\{z/b, x/f(y), y/g(z)\}$

2) Find most general Unifier (MGU) of $\{Q(a, g(x, a), f(y)), Q(a, g(f(b), a), x)\}$

$\Rightarrow$

- $a = a$
- $g(x, a) = g(f(b), a) \rightarrow x = f(b)$
- $f(y) = x \rightarrow f(y) = f(b)$
- $x \rightarrow f(b)$
- $y \rightarrow b$

MGU: $\{x/f(b), y/b\}$

3) Find MGU of $\{P(f(a), g(y)), P(x, x)\}$

$\Rightarrow$

- $f(a) = x$
- $g(y) = x$
- $f(a) = g(y) \rightarrow$ which cannot unify

$\therefore$ Therefore, no unifier exists.

4) Unify $\{ \text{prime}(11) \text{ \& } \text{prime}(y) \}$
 $\Rightarrow$ $\cdot 11$ is a constant, y is a variable
 $\cdot \text{prime}(11) = \text{prime}(y) \rightarrow y = 11$
 Mgu: $\{ y/11 \}$

5) Unify $\{ \text{knows}(\text{John}, x), \text{knows}(y, \text{mother}(y)) \}$
 $\Rightarrow$ $\cdot \text{knows John} = y \rightarrow y = \text{John}$
 $\cdot x = \text{mother}(y) \rightarrow x = \text{mother}(\text{John})$
 Mgu: $\{ y/\text{John}, x/\text{mother}(\text{John}) \}$

6) Unify $\{ \text{knows}(\text{John}, x), \text{knows}(y, \text{Bill}) \}$
 $\Rightarrow$ $\cdot \text{knows John} = y \rightarrow y = \text{John}$
 $\cdot x = \text{Bill}$
 Mgu: $\{ y/\text{John}, x/\text{Bill} \}$

Unification Algorithm.

step 1: If ψ_1 or ψ_2 is a variable or constant, then:

a) If ψ_1 or ψ_2 are identical, then return NIL

b) Else if ψ_1 is a variable,

a. then if ψ_1 occurs in ψ_2 then return FAILURE

b. Else return $\{ \psi_2 / \psi_1 \}$

c) Else if ψ_2 is a variable,

a. if ψ_2 occurs in ψ_1 then return FAILURE,

b. Else return $\{ \psi_1 / \psi_2 \}$

d) Else return FAILURE

step 2: If the initial predicate symbol in ψ_1 & ψ_2 are not same, then return FAILURE

step 3: If ψ_1 & ψ_2 have a different number of arguments, then return FAILURE

step 4: Set substitution set (SUBSET) to NIL

step 5: For $i = 1$ to the number of elements in ψ_1

- a) Call Omgly function with i th element of Y_1 & j th element of Y_2 , & put the result into S
 - b) If $S = \text{failure}$ then returns Failure
 - c) If $S \neq \text{NIL}$ then do,
 - a. Apply S to the remainder of both L_1 & L_2
 - b. $\text{SUBST} = \text{APPEND}(S, \text{SUBST})$
- Step 6: Return SUBST

O/P :
 Mho : $\langle 'b' : 'z', 'x' : ('j', 'y'), 'y' : ('g', 'z') \rangle$

MGU: {'b': 'z', 'x': ('f', 'Y'), 'Y': ('g', 'Z')}

Srushti N
1BM24CS424

Code:

```
def occurs_check(var, expr):
    if var == expr:
        return True
    if isinstance(expr, tuple):
        return any(occurs_check(var, sub) for sub in expr[1:]) # Skip function symbol
    return False

def substitute(expr, subst):
    if isinstance(expr, str):
        # Follow substitution chain until fully resolved
        while expr in subst:
            expr = subst[expr]
        return expr
    # If it's a function term: (f, arg1, arg2, ...)
    return (expr[0],) + tuple(substitute(sub, subst) for sub in expr[1:])

def unify(Y1, Y2, subst=None):
    if subst is None:
        subst = {}
    Y1 = substitute(Y1, subst)
    Y2 = substitute(Y2, subst)

    # Case 1: identical
    if Y1 == Y2:
        return subst

    # Case 2: Y1 is variable
    if isinstance(Y1, str):
        if occurs_check(Y1, Y2):
            return "FAILURE"
        subst[Y1] = Y2
        return subst
```

```

# Case 3: Y2 is variable
if isinstance(Y2, str):
    if occurs_check(Y2, Y1):
        return "FAILURE"
    subst[Y2] = Y1
    return subst

# Case 4: function mismatch
if Y1[0] != Y2[0] or len(Y1) != len(Y2):
    return "FAILURE"

# Case 5: unify arguments
for a, b in zip(Y1[1:], Y2[1:]):
    subst = unify(a, b, subst)
    if subst == "FAILURE":
        return "FAILURE"

return subst

expr1 = ("p", "X", ("f", "Y"))
expr2 = ("p", "a", ("f", "b"))

output = unify(expr1, expr2)
print(output)

```

OUTPUT:

```
{'X': 'a', 'Y': 'b'}
```

Program 8

Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.

Algorithm:

Page No:
 Date: 13 10 25

Week - 8
First Order Logic

Q. Create a knowledge base consisting of first order logic statement & prove the given query using forward reasoning

Premises

Rules

Facts

Conclusion

$$\begin{aligned}
 &L \wedge M \Rightarrow P \\
 &B \wedge L \Rightarrow M \\
 &A \wedge P \Rightarrow L \\
 &A \wedge B = L
 \end{aligned}$$

A

B

Prove Q

```

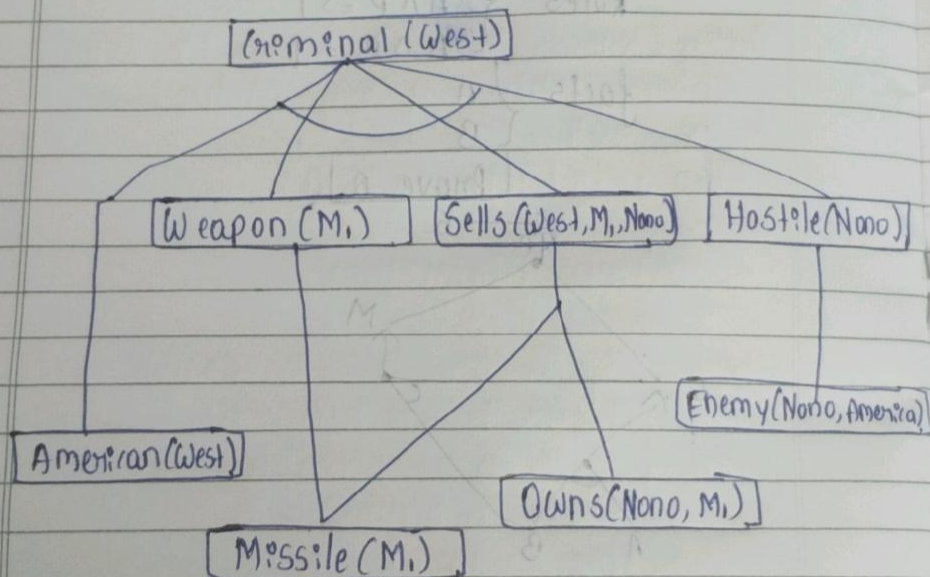
graph BT
    A((A)) --- L((L))
    B((B)) --- L
    L --- P((P))
    P --- M((M))
    M --- Q((Q))
    style Q fill:none,stroke:none
  
```

Q. The law says that it is a crime for an American to sell weapons to hostile nations. The Country Nono an enemy of America, has some missiles & all of its missiles were sold to it by Colonel West who is American. An enemy of America counts as 'hostile' Prove that "West is Criminal"

=> Rules

1) $\forall x, y, z \text{ American}(x) \wedge \text{weapon}(y) \wedge$

- Rules
- 1) $\forall x \text{ Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$
 - 2) $\forall x \text{ Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$
 - 3) $\forall x \text{ Missile}(x) \Rightarrow \text{Weapon}(x)$
 - 4) $\text{American}(\text{West})$
 - 5) $\text{Enemy}(\text{Nono}, \text{America})$
 - 6) $\text{Owns}(\text{Nono}, M_1)$ and
 - 7) $\text{Missile}(M_1)$
- Facts



Algorithm

function FOL-FC-Ask(KB, α) returns a substitution or false
 inputs: KB , the knowledge base, a set of first-order definite clauses
 α , the query, an atomic sentence
 local variables: new, the new sentences inferred on each iteration

Repeat until new is empty

new $\leftarrow \langle \rangle$

for each rule in KB do

$(P_1 \wedge \dots \wedge P_n \Rightarrow Q) \leftarrow \text{STANDARDIZE-VARIABLES}(\text{rule})$

for each θ such that $\text{SUBST}(\theta, P_1, \dots, P_n) =$
 $\text{SUBST}(\theta, P'_1 \wedge \dots \wedge P'_n)$

for some $P'_1, \dots, P'_n$ in KB

$Q' \leftarrow \text{SUBST}(\theta, Q)$

if Q' does not unify with some sentence
 already in KB on new then

add Q' to new

$\theta \leftarrow \text{UNIFY}(Q', \alpha)$

if θ is not fail then return ϕ

add new to KB

return false.

Output:

Adding fact: American(West)

Adding fact: Enemy(Nono, America)

Adding fact: Missile(M1)

Adding fact: Owns(Nono, M1)

Inferred new fact: Hostile(Nono) from

$['\text{Enemy}(\text{Nono}, \text{America})'] \Rightarrow \text{Hostile}(\text{Nono})$

Inferred new fact: Weapon(M1) from $['\text{Missile}(\text{M1})'] \Rightarrow \text{Weapon}(\text{M1})$

Inferred new fact: Sells(West, M1, Nono) from
 $['\text{Missile}(\text{M1})', '\text{Owns}(\text{Nono}, \text{M1})'] \Rightarrow \text{Sells}(\text{West}, \text{M1}, \text{Nono})$

Inferred new fact: Criminal(West) from $['\text{American}(\text{West})', '\text{Weapon}(\text{M1})', '\text{Sells}(\text{West}, \text{M1}, \text{Nono})', '\text{Hostile}(\text{Nono})'] \Rightarrow$
 Criminal(West)

Goal Reached: West is criminal
 True.

```
Adding fact: American(West)
Adding fact: Enemy(Nono, America)
Adding fact: Missile(M1)
Adding fact: Owns(Nono, M1)
Inferred new fact: Hostile(Nono) from ['Enemy(Nono, America)'] => Hostile(Nono)
Inferred new fact: Weapon(M1) from ['Missile(M1)'] => Weapon(M1)
Inferred new fact: Sells(West, M1, Nono) from ['Missile(M1)', 'Owns(Nono, M1)'] => Sells(West, M1, Nono)
Inferred new fact: Criminal(West) from ['American(West)', 'Weapon(M1)', 'Sells(West, M1, Nono)', 'Hostile(Nono)'] => Criminal(West)

Goal Reached: West is Criminal
True
```

Srushti N
18M24CS424

Code:

```
from collections import deque

class KnowledgeBase:

    def __init__(self):
        self.facts = set()
        self.rules = []
        self.inferred = set()

    def add_fact(self, fact):
        if fact not in self.facts:
            print(f'Adding fact: {fact}')
            self.facts.add(fact)
            return True
        return False

    def add_rule(self, premises, conclusion):
        self.rules.append((premises, conclusion))

    def forward_chain(self):
        agenda = deque(self.facts)

        while agenda:
            fact = agenda.popleft()
            if fact in self.inferred:
                continue
            self.inferred.add(fact)

            for (premises, conclusion) in self.rules:
                if all(p in self.inferred for p in premises):
                    if conclusion not in self.facts:
```

```

        print(f"Inferred new fact: {conclusion} from {premises} => {conclusion}")
        self.facts.add(conclusion)
        agenda.append(conclusion)

    if conclusion == 'Criminal(West)':
        print("\n☑ Goal Reached: West is Criminal")
        return True
    return False

kb = KnowledgeBase()

kb.add_fact('American(West)')
kb.add_fact('Enemy(Nono, America)')
kb.add_fact('Missile(M1)')
kb.add_fact('Owns(Nono, M1)')

kb.add_rule(premises=['Missile(M1)'], conclusion='Weapon(M1)')

kb.add_rule(premises=['Missile(M1)', 'Owns(Nono, M1)'], conclusion='Sells(West, M1, Nono)')

kb.add_rule(premises=['Enemy(Nono, America)'], conclusion='Hostile(Nono)')

kb.add_rule(premises=['American(West)', 'Weapon(M1)', 'Sells(West, M1, Nono)', 'Hostile(Nono)'],
conclusion='Criminal(West)')

kb.forward_chain()

```

OUTPUT:

```

Adding fact: American(West)
Adding fact: Enemy(Nono, America)
Adding fact: Missile(M1)
Adding fact: Owns(Nono, M1)
Inferred new fact: Weapon(M1) from ['Missile(M1)'] => Weapon(M1)
Inferred new fact: Hostile(Nono) from ['Enemy(Nono, America)'] => Hostile(Nono)
Inferred new fact: Sells(West, M1, Nono) from ['Missile(M1)', 'Owns(Nono, M1)'] => Sells(West, M1, Nono)
Inferred new fact: Criminal(West) from ['American(West)', 'Weapon(M1)', 'Sells(West, M1, Nono)', 'Hostile(Nono)'] => Criminal(West)
☑ Goal Reached: West is Criminal
True

```

Program 9

Create a knowledge base consisting of first order logic statements and prove the given query using Resolution

Algorithm:

Page No:

Date: 07 10 25

Week - 09
First Order Logic

Create a knowledge base consisting of FOL statements and prove the given query using Resolution.

Proof by Resolution

Given KB or premises

- John likes all kind of food
- Apple & vegetables are food
- Anything anyone eats & not killed is food
- Anil eats peanuts & still alive
- Harry eats everything that Anil eats
- Anyone who is alive implies not killed
- Anywh Anyone who is not killed implies alive.

Prove by Resolution that:
John likes peanut

Representation in FOL

- a. $\forall x : \text{food}(x) \rightarrow \text{likes}(\text{John}, x)$
- b. $\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$
- c. $\forall x \forall y : \text{eats}(x, y) \wedge \neg \text{killed}(x) \rightarrow \text{food}(y)$
- d. $\text{eats}(\text{Anil}, \text{peanuts}) \wedge \text{alive}(\text{Anil})$
- e. $\forall x : \text{eats}(\text{Anil}, x) \rightarrow \text{eats}(\text{Harry}, x)$
- f. $\forall x : \neg \text{killed}(x) \rightarrow \text{alive}(x)$
- g. $\forall x : \text{alive}(x) \rightarrow \neg \text{killed}(x)$
- h. $\text{likes}(\text{John}, \text{peanuts})$

Eliminate implication

- $\forall x \neg \text{food}(x) \vee \text{likes}(\text{John}, x)$
- $\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$
- $\forall x \forall y \neg [\text{eats}(x, y) \wedge \neg \text{killed}(x)] \vee \text{food}(y)$
- $\text{eats}(\text{Anil}, \text{peanuts}) \wedge \text{alive}(\text{Anil})$
- $\forall x \neg \text{eats}(\text{Anil}, x) \vee \text{eats}(\text{Harry}, x)$
- $\forall x \neg [\neg \text{killed}(x)] \vee \text{alive}(x)$
- $\forall x \neg \text{alive}(x) \vee \neg \text{killed}(x)$
- $\text{likes}(\text{John}, \text{peanuts})$

Move negation ($\neg$) inwards & rewrite

- $\forall x \neg \text{food}(x) \vee \text{likes}(\text{John}, x)$
- $\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$
- $\forall x \forall y \neg \text{eats}(x, y) \vee \text{killed}(x) \vee \text{food}(y)$
- $\text{eats}(\text{Anil}, \text{peanuts}) \wedge \text{alive}(\text{Anil})$
- $\forall x \neg \text{eats}(\text{Anil}, x) \vee \text{eats}(\text{Harry}, x)$
- $\forall x \text{killed}(x) \vee \text{alive}(x)$
- $\forall x \neg \text{alive}(x) \vee \neg \text{killed}(x)$
- $\text{likes}(\text{John}, \text{peanuts})$

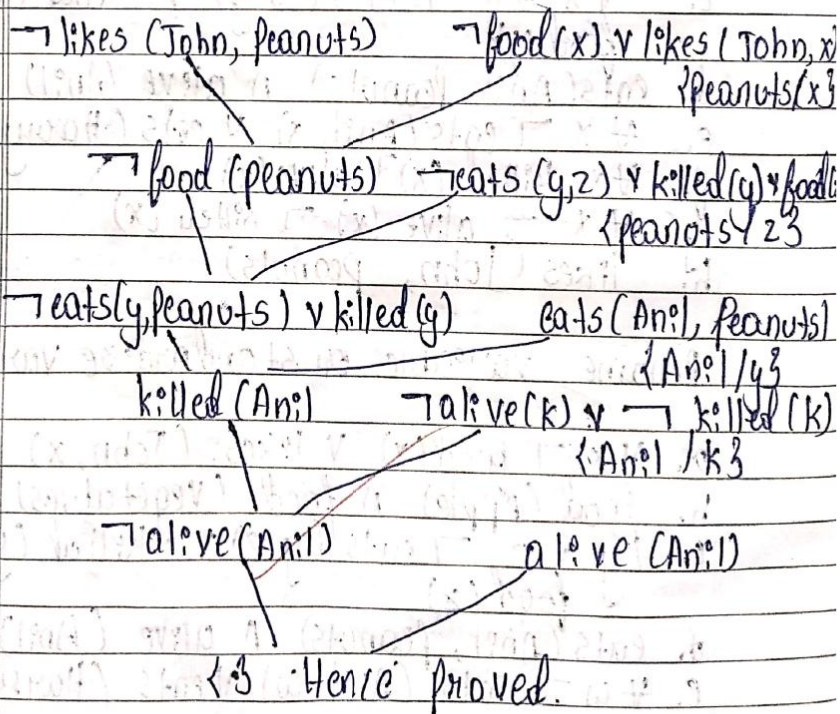
Rename variable on standardize variables

- $\forall x \neg \text{food}(x) \vee \text{likes}(\text{John}, x)$
- $\text{food}(\text{Apple}) \wedge \text{food}(\text{vegetables})$
- $\forall y \forall z \neg \text{eats}(y, z) \vee \text{killed}(y) \vee \text{food}(z)$
- $\text{eats}(\text{Anil}, \text{peanuts}) \wedge \text{alive}(\text{Anil})$
- $\forall w \neg \text{eats}(\text{Anil}, w) \vee \text{eats}(\text{Harry}, w)$

- f. $\neg g$ killed (g) $\vee$ alive (g)
 g. $\forall k \neg$ alive(k) $\vee \neg$ killed(k)
 h. likes (John, Peanuts)

Drop universe

- a. $\neg$ food (x) $\vee$ likes (John, x)
 b. food (Apple)
 c. food (vegetables)
 d. $\neg$ eats (g, z) $\vee$ killed (y) $\vee$ food (z)
 e. eats (Anil, Peanuts).
 f. alive (Anil)
 g. $\neg$ eats (Anil, w) $\vee$ eats (Harvey, w)
 h. killed (g) $\vee$ alive (g)
 i. $\neg$ alive (k) $\vee \neg$ killed (k)
 j. likes (John, Peanuts)



Week - 10

Algorithm.

1. Input:

- Knowledge Base (KB)

- Query (Q)

2. Convert KB and $\neg Q$ to clausal Form:

- Eliminate implications.

- Move negations inward

- Standardize variables

- Skolemize (remove $\exists$ quantifiers)

- Drop universal quantifiers

- Convert to CNF

3. Apply Resolution:

- Repeatedly resolve pairs of clauses that contain complementary literals.

- Add new clauses to the KB

- Stop if

- Empty clause (1) is derived $\rightarrow Q$ is true

- No new clauses can be added $\rightarrow Q$ is False

4. Output True / False for Query Q.

Code:

```
from itertools import combinations

def get_clauses():
    n = int(input("Enter number of clauses in Knowledge Base: "))
    clauses = []
    for i in range(n):
        clause = input(f"Enter clause {i+1}: ")
        clause_set = set(clause.replace(" ", "").split("v"))
        clauses.append(clause_set)
    return clauses

def resolve(ci, cj):
    resolvents = []
    for di in ci:
        for dj in cj:
            if di == ('~' + dj) or dj == ('~' + di):
                new_clause = (ci - {di}) | (cj - {dj})
                resolvents.append(new_clause)
    return resolvents

def resolution_algorithm(kb, query):
    kb.append(set(['~' + query]))
    derived = []
    clause_id = {frozenset(c): f"C{i+1}" for i, c in enumerate(kb)}

    step = 1
    while True:
        new = []
```

```

for (ci, cj) in combinations(kb, 2):
    resolvents = resolve(ci, cj)
    for res in resolvents:
        if res not in kb and res not in new:
            cid_i, cid_j = clause_id[frozenset(ci)], clause_id[frozenset(cj)]
            clause_name = f'R{step}'
            derived.append((clause_name, res, cid_i, cid_j))
            clause_id[frozenset(res)] = clause_name
            new.append(res)
            print(f'[Step {step}] {clause_name} = Resolve({cid_i}, {cid_j}) → {res or '{}'}')
            step += 1

    # If empty clause found → proof complete
    if res == set():
        print("\n☑ Query is proved by resolution (empty clause found).")
        print("\n--- Proof Tree ---")
        print_tree(derived, clause_name)
        return True

    if not new:
        print("\n✗ Query cannot be proved by resolution.")
        return False

    kb.extend(new)

def print_tree(derived, goal):
    tree = {name: (parents, clause) for name, clause, *parents in [(r[0], r[1], r[2:][0], r[2:][1]) for r in
derived]}

def show(node, indent=0):
    if node not in tree:
        print(" " * indent + node)

```



```

    return

    parents, clause = tree[node]

    print(" " * indent + f"{node}: {set(clause) or '{}'}")

    for p in parents:
        show(p, indent + 4)

show(goal)

```

OUTPUT:

```

=== FOL Resolution Demo with Proof Tree ===
Enter number of clauses in Knowledge Base: 3
Enter clause 1: P
Enter clause 2: ~P v Q
Enter clause 3: ~Q
Enter query to prove: Q
[Step 1] R1 = Resolve(C1, C2) → {'Q'}
[Step 2] R2 = Resolve(C2, C4) → {'~P'}
[Step 3] R3 = Resolve(C1, R2) → {}

✅ Query is proved by resolution (empty clause found).

--- Proof Tree ---
R3: {}
   C1
   R2: {'~P'}
      C2
      C4

True

```

Program 10

Implement Alpha-Beta Pruning.

Algorithm:

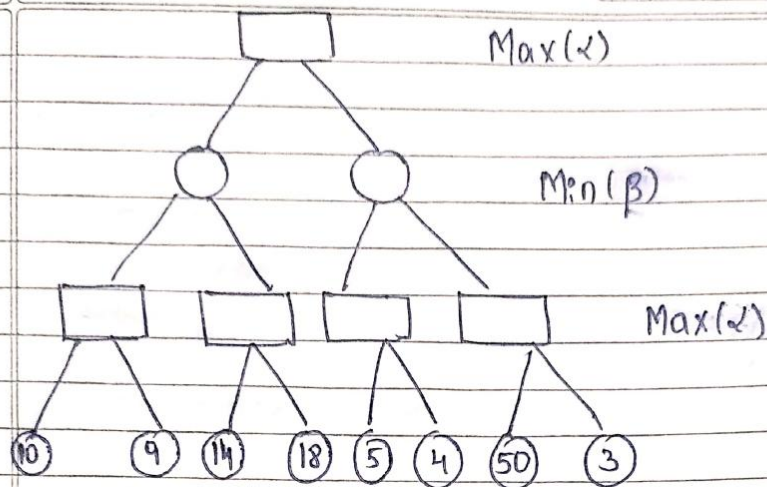
Page No:

Date: / /

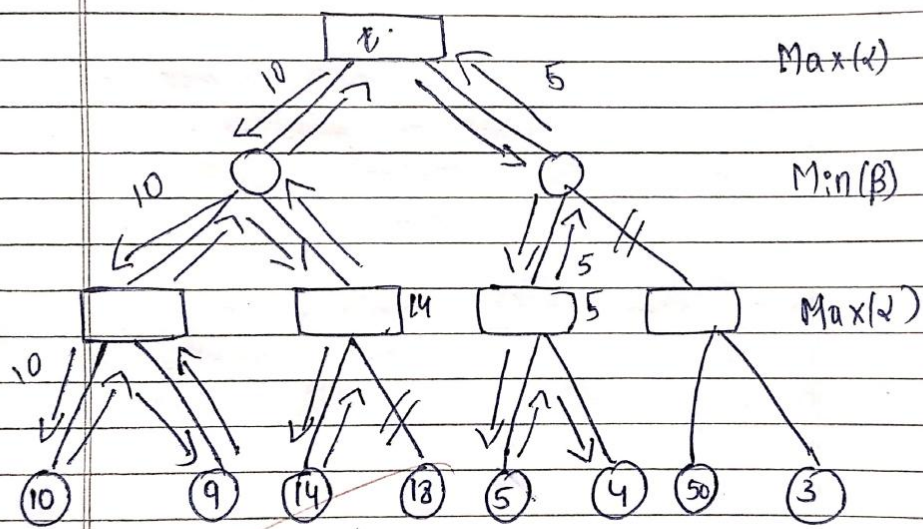
Week - 10
Adversarial Search
Implement Alpha - Beta Pruning.

Algorithm :

1. Start at the root node (current game start)
The current player is either max or min
2. Initialize
 - $\alpha = -\infty$
 - $\beta = +\infty$
3. If terminal node (end of game):
 - Return the utility (score) of that node
4. If it's a max player:
 - Set value = $-\infty$
 - For each child of this node:
 - 1) Compute child value = $\text{AlphaBeta}(\text{child}, \text{depth} - 1, \alpha, \beta, \text{False})$
 - 2) update value = $\max(\text{value}, \text{child_value})$
 - 3) update $\alpha = \max(\text{value}, \text{child_value})$
 - 4) if $\alpha \geq \beta$, then break → (prune remaining branches).
 - Return value.
5. If it's a min player:
 - Set value = $+\infty$
 - For each child value =
 - 1) $\text{AlphaBeta}(\text{child}, \text{depth} - 1, \alpha, \beta, \text{True})$
 - 2) update value = $\min(\text{value}, \text{child_value})$
 - 3) update $\beta = \min(\beta, \text{value})$
 - 4) if $\alpha \geq \beta$, then break → (prune remaining branches)
 - Return value.



Solution:



Handwritten signature
27-10-22

Code:

class Node:

```
def __init__(self, name):
    self.name = name
    self.children = []
    self.value = None
    self.pruned = False
```

def alpha_beta(node, depth, maximizing, values, alpha, beta, index):

Terminal node

if depth == 3:

node.value = values[index[0]]

index[0] += 1

return node.value

if maximizing:

best = float('-inf')

for i in range(2): # 2 children

child = Node(f"{node.name} {i}")

node.children.append(child)

val = alpha_beta(child, depth + 1, False, values, alpha, beta, index)

best = max(best, val)

alpha = max(alpha, best)

if beta <= alpha:

node.pruned = True

break

node.value = best

return best

else:

best = float('inf')

```

    for i in range(2):
        child = Node(f"{node.name} {i}")
        node.children.append(child)
        val = alpha_beta(child, depth + 1, True, values, alpha, beta, index)
        best = min(best, val)
        beta = min(beta, best)
        if beta <= alpha:
            node.pruned = True
            break
    node.value = best
    return best

def print_tree(node, indent=0):
    prune_mark = " [PRUNED]" if node.pruned else ""
    val = f" = {node.value}" if node.value is not None else ""
    print(" " * indent + f"{node.name} {val} {prune_mark}")
    for child in node.children:
        print_tree(child, indent + 4)

# --- main ---
print("=== Alpha-Beta Pruning with Tree ===")
values = list(map(int, input("Enter 8 leaf node values separated by spaces: ").split()))

root = Node("R")
alpha_beta(root, 0, True, values, float('-inf'), float('inf'), [0])

print("\n--- Game Tree ---")
print_tree(root)

print("\nOptimal Value at Root:", root.value)

```


OUTPUT:

```
=== Alpha-Beta Pruning with Tree ===  
Enter 8 leaf node values separated by spaces: 3 5 6 9 1 2 0 7  
  
--- Game Tree ---  
R = 5  
  R0 = 5  
    R00 = 5  
      R000 = 3  
      R001 = 5  
    R01 = 6 [PRUNED]  
      R010 = 6  
  R1 = 2 [PRUNED]  
    R10 = 9  
      R100 = 9  
      R101 = 1  
    R11 = 2  
      R110 = 2  
      R111 = 0  
  
Optimal Value at Root: 5
```